

**AI took my job,
thankfully**

A decorative graphic consisting of a thin orange arc that curves from the top right towards the bottom right, with a solid orange circle positioned near the end of the arc.

Tech Passion Day 2026 · Copenhagen · 16 September 2026

Which job, exactly?

What AI took

- First drafts
- Boilerplate
- Syntax lookups
- The third config file
- Code that already exists somewhere

What it left

- What to build
- Understanding it
- The trade-offs
- The accountability

What nobody on this stage can promise



Entry-level hiring has tightened



Roles are changing faster than titles



Nobody can promise your job is safe

The skills that matter now
can be learned.

REACH

The workday got wider

AI expands the volume and variety of work we can attempt.

HIRED FOR

Backend service code

CAN NOW ATTEMPT

Migration script

Terraform module

Load-test harness

Runbook

Frontend prototype

Data pipeline

More attempts, more drafts, more decisions



Attempt



Draft



Decide



Ship

The slow part moved from typing to judging.

More reach means more to answer for



Is it correct?



Is it safe?



Does it belong?



Can we run it?



Should it exist at all?

Whatever you keep,
you now own.

Delegation is not abdication

Delegation

- ✓ Say what you want first
- ✓ Small pieces, checked one at a time
- ✓ Read it and run it before merging
- ✓ Explain any line you kept

Abdication

- ✗ Tests it wrote for itself
- ✗ Output you don't understand
- ✗ Rewording instead of understanding
- ✗ A green pipeline as the only reviewer

Fundamentals make the review fast



Data and consistency

Will this race under load?



Networks and HTTP

What happens on the retry?



Security

Who can call this, and with what?



Concurrency

Where does this block?



Cost and capacity

What happens at 100× traffic?



Testing

What does this test actually prove?

Durable knowledge outlives your tools

Ages in decades

- Transactions and consistency
- TCP and HTTP
- Relational modelling
- Queues and backpressure
- Threat modelling
- How distributed systems fail

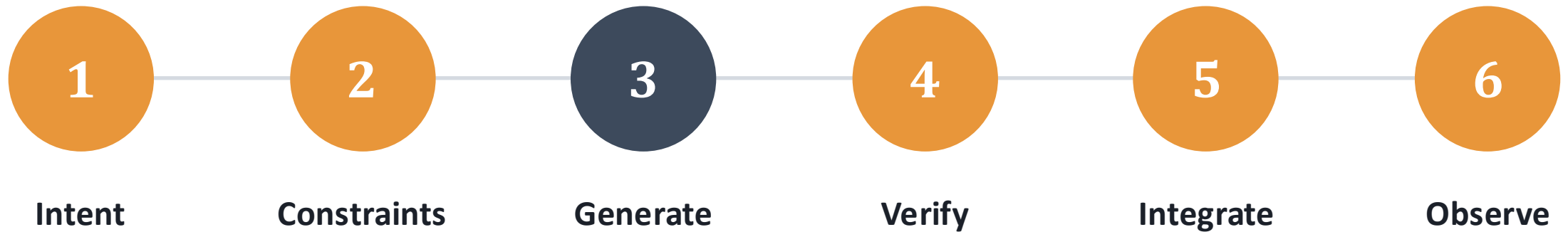
Ages in months

- This year's framework API
- A vendor's console
- One model's quirks
- The prompt trick from March

Your existing knowledge already transfers

You already do		It becomes
Code review	→	Reviewing generated changes
Writing a clear spec	→	Writing intent and constraints
Test-driven development	→	Eval-driven development
Observability	→	Tracing what an agent did
Threat modelling	→	Deciding what an agent may touch
Incident response	→	A model is another failing dependency
Mentoring	→	Briefing a collaborator with no context

A practice for AI-assisted engineering



The tool does step 3. You do the rest.

A worked example

Replace a nightly batch job with an event-driven pipeline.

Step	The tool drafts	You decide
1 Intent	Summarises the old job	Which events, what latency, what if the consumer is down
2 Constraints	Lists the options	Idempotency, ordering, retention, ownership
3 Generate	Consumer, laC, tests, runbook	One component at a time
4 Verify	A first pass at tests	Does the retry duplicate writes? Where are the secrets?
5 Integrate	PR text, draft ADR	How to split it, and why
6 Observe	Dashboards, alerts	Lag, error rate, cost

Where it goes wrong



Merged without reading

Explain every line you keep.



Tests from the same model

Write the assertions yourself.



Prompting past confusion

Go learn the concept.



Big-bang generation

Ten minutes to review, or it's too big.



Observability later

Ship the alerts with the feature.



The tool sets the architecture

You draw the boundaries.

If you are starting out

The easy first tasks are the ones the tools do first.

You can ship complete things earlier than anyone before you.

- 1 Build end to end
- 2 Read every line before you keep it
- 3 One fundamental per project
- 4 Practise explaining why

If you are a few years in



Take the whole problem

Including the parts outside your title.



Start in unfamiliar territory

Then check it with what you already know.



Review across boundaries

Backend, infra and data in one afternoon.



Write the decisions down

The decision record outlasts the code.

If you lead a team or an architecture



Say what reviewed means



Design for fast change



Least privilege for tools too



Ask people to explain their changes

Leverage used to be how much you could write.

Now it is how much you can safely let others attempt.

“

You can become capable of more than your current job requires. The knowledge you build while doing that can travel with you.

NEXT STEP

What travels with you

YOURS

- ✓ Fundamentals
- ✓ Checking before trusting
- ✓ The domain
- ✓ Explaining a decision
- ✓ Work you can point to

ON LOAN

- A vendor's console
- A prompt library
- A job title
- This quarter's tooling

Find your gap

1

Open the last change you shipped with AI in it

The last one. Not your best one.

2

Ask the six questions about it, from memory

Race. Retry. Caller. Blocking. 100x. Tests.

3

Every blank is debt

Pay one down before you generate anything else.

AI raised the ceiling on what you can attempt.
It did not lower the floor on what you have to understand.
The gap between them is either debt or growth.

Thank you

Questions?

"You can become capable of more than your current job requires. The knowledge you build while doing that can travel with you."